

When *not* to use AIDA

AIDA positioning · best-effort comparison — live source:
github.com/joemooney/aida

rendered 2026-07-10

Last updated: 2026-06-09

Every honest tool has a shape it doesn't fit. AIDA's machinery — a git-canonical requirement graph, stable IDs, typed relationships, code↔spec traces, an orchestrated lifecycle, an MCP server — is overhead. That overhead pays for itself when value **compounds**: across many specs, many sessions, many agents, many months, many hands. Below that threshold it's cost without the payoff.

This page names the cases where you should reach for something else, and what. Telling you when *not* to adopt AIDA is the cheapest way we have to earn your trust the rest of the time.

The one-line test: *Does value compound in your project — multiple agents, multiple sessions, cross-month continuity, multi-developer handoffs?* If yes, AIDA's graph earns its keep. If no, one of the cases below is a better fit.

1. Solo developer, one-off or short-lived work, no agents

If you're one person building something small that won't outlive the month, and you're writing the code yourself, the graph never gets big enough to pay for itself. You'll spend more time filing specs than you save querying them.

Use instead: plain git plus a `TODO.md`, or — if you want first-feature structure — [GitHub Spec Kit](#) to scaffold a spec → plan → tasks flow with zero infrastructure. AIDA's added machinery (cache, orphan branch, MCP server, queue, lifecycle) wouldn't earn its keep at that scale.

AIDA starts to be worth it the moment any of these becomes true: a coding agent is in the loop, the project spans more than a couple of sessions, or a second person has to pick up where you left off.

2. Single-vendor shop with no portability needs

AIDA's load-bearing differentiator is that the requirement graph is **git-canonical and vendor-neutral** — the same store serves Claude Code, Codex, Cursor, Continue, and a plain CLI, and outlives any one of them. If you are all-in on a single agent and will never run a second vendor against the same specs, that differentiator doesn't pay.

Use instead: your agent's native coordination, layered on Spec Kit. Claude Code's **Agent Teams** ships a native mailbox, a shared self-claiming task list, dependency auto-unblocking, and a plan-approval gate — within a session, in one vendor's tooling, with zero extra infrastructure. For a single-vendor team that question ("multi-vendor?") is the whole decision: answer "no" and Agent Teams + Spec Kit is the lighter, sufficient choice.

AIDA's bet is that the graph should outlive the tool that wrote it. If you don't share that bet, don't pay for it.

3. Real-time, many-humans-at-once collaboration

AIDA's substrate is git: changes land as commits and merge through the orphan aida-store branch. That's durable and diffable, but it is **coarse-grained** — it is not a live, multi-cursor surface. Ten people editing the same specs simultaneously, watching each other's changes stream in, is not what a git model does well.

Use instead: Linear, Jira, Notion, or GitHub Projects for the live board and real-time editing. AIDA can mirror to and from GitHub Issues (`aida github push / pull`) if you want the durable graph *and* a live board — but the live board itself should be the SaaS tool. See [vs-saas-pm.md](#) for that split in full.

4. Conversational or semantic memory

AIDA remembers **specs and traces** — typed requirements, their relationships, and the code that implements them. It does not remember conversations. "What did we discuss about caching three weeks ago?" or "find the message where someone mentioned the rate-limit bug" is a fuzzy-recall, vector-search problem.

Use instead: a semantic-memory layer (mem0, Letta, and similar) for free-text conversational recall. They are complementary, not competing: AIDA is the *structured* memory (the graph of intent), a vector store is the *associative* memory (the haystack of conversation). Reaching for AIDA to recall a chat message is using a filing cabinet as a search engine.

5. You want zero discipline, not just zero infrastructure

AIDA is low-infrastructure (one binary, a git branch, a local cache) but it is **not zero-discipline**. It asks for a habit: file the spec, drop the one-line `// trace: comment`, name the spec in the commit. The payoff — `aida show` telling you *why this code exists* six months later — only exists because that habit fed it.

If you won't maintain even a structured `REQUIREMENTS.md` by hand, AIDA's graph won't maintain itself either. The “**structured markdown an agent can read**” approach is the floor; AIDA adds the relationship graph, stable IDs, and enforcement *on top of* a discipline you're already willing to keep. No discipline, no graph.

Use instead: nothing — or a single hand-kept markdown file — until the habit is something you actually want. AIDA rewards the discipline; it can't supply it.

6. A pure documentation site or knowledge base

If what you want is a published handbook, API reference, or wiki — prose for humans, not a graph of intent linked to code — AIDA is the wrong layer.

Use instead: `mdBook`, `DocuSaurus`, or a static-site generator. AIDA projects to documentation (`aida docs build` renders the graph into linkable pages) but it is not a substitute for a docs platform. Use the docs tool for the published surface; use AIDA if you also want that surface tied back to live specs and code.

The honest frame

AIDA's value curve is **super-linear in collaborators, sessions, and time** — and roughly break-even-to-negative below the threshold where those start to accumulate. The machinery that feels like ceremony on day one (stable IDs, typed edges, traces, lifecycle) is exactly what stops “why did we choose X?” from being re-debated, cross-references from rotting, and a cold-starting agent from re-deriving the whole project every session.

So the decision isn't “is AIDA good?” — it's “does my project sit above or below the line where a maintained graph pays for itself?” If you're above it (agents in the loop, work spanning sessions and people and months), AIDA is built for you. If you're below it, one of the six tools above will serve you better today — and AIDA will still be here when your project grows into it.

See also

- **How AIDA compares** — the full one-neighbor-at-a-time index.
- **vs Spec Kit** — AIDA's nearest competitor; the clearest “scaffold first-feature vs maintain the cross-cutting graph” contrast.

- **vs Claude Code Agent Teams** — single-vendor native coordination vs cross-vendor durable substrate.
- **Composition recipes** — when the answer is “use AIDA *with* X,” not “instead of.”